Reliability Engineering and System Safety 94 (2009) 1703–1710

Contents lists available at ScienceDirect

Reliability Engineering and System Safety

journal homepage: www.elsevier.com/locate/ress

# An efficient heuristic versus a robust hybrid meta-heuristic for general framework of serial–parallel redundancy problem ☆

Seyed Jafar Sadjadi *, R. Soltani

*Department of Industrial Engineering, Iran University of Science and Technology, Tehran, Iran*

ARTICLE INFO

*Article history:*
Received 15 January 2009
Received in revised form
18 March 2009
Accepted 10 May 2009
Available online 18 May 2009

*Keywords:*
Redundancy
Heuristic
Reliability
Hybrid genetic algorithm
Taguchi method

ABSTRACT

We present a heuristic approach to solve a general framework of serial–parallel redundancy problem where the reliability of the system is maximized subject to some general linear constraints. The complexity of the redundancy problem is generally considered to be NP-Hard and the optimal solution is not normally available. Therefore, to evaluate the performance of the proposed method, a hybrid genetic algorithm is also implemented whose parameters are calibrated via Taguchi's robust design method. Then, various test problems are solved and the computational results indicate that the proposed heuristic approach could provide us some promising reliabilities, which are fairly close to optimal solutions in a reasonable amount of time.

© 2009 Elsevier Ltd. All rights reserved.

## 1. Introduction

During the past decades, there have been tremendous progresses on high-tech industries for designing reliable chips. The reliability has been a key issue especially when we are designing problems for military facilities, space shuttle, etc. The primary objective in these areas is to build a highly reliable tool. Therefore, one may be interested in having more than one component in parallel where different particles are working in serial just to make sure that the failure of one single component would not result to the failure of the entire system. The serial parallel system could be considered from three different perspectives: the reliability allocation, the redundancy allocation and the reliability and the redundancy allocation. In the reliability allocation problems, the reliability of the components is determined such that the consumption of a resource under a reliability constraint is minimized while the redundancy allocation problem generally involves the selection of components and redundancy levels to maximize the system reliability given various system-level's constraints. The reliability and the redundancy allocation, which is the combination of two aforesaid problems determine the reliability of the components and the redundancy levels simultaneously in order to minimize the consumption of a resource under a reliability constraint. This study is focused on the redundancy allocation problems, which are normally involved with real-world applications of more than one constraint. Namely, increasing the system's components often deals with many restrictions such as the limit on the budget, weight, volume, etc. These restrictions often make the mathematical formulation even more complicated. Most of the existing mathematical programming applications in the literature are presently focused only on one or two constraints and we could hardly find any relative work on problems with three or more constraints. The relative literature reviews considering three aforementioned categories are as follows:

Regarding the reliability allocation category, Elegbede et al. [1] introduce a simple cost function for reliability allocation and propose a new method to find the optimal solution. Yalaoui et al. [2] study the reliability allocation problem in different form. They study a specific form of series–parallel when each subsystem is composed of different components in series and all serial systems are connected in parallel form. Yalaoui et al. [2] develop some mathematical models for their proposed system, which ends to be a minimization of a non-convex function subject to some nonlinear constraints and the optimal solution for the resulted problem is found using some approximations.

The mathematical formulation of the redundancy allocation is considered in different forms [3]. Misra [4] considers the maximization of the reliability for the whole system. Ramirez-Marquez et al. [5,6] reformulate the redundancy problem using a max–min approach. They make some modifications on the original redundancy problem and change it into a mixed integer programming. The resulted formulation is modified and solved for a near-optimal solution. Tavakkoli-Moghaddam et al. [7] study active and cold-standby redundancy within a particular system design. They propose a genetic algorithm to find the near-optimal solution and, using some benchmark problems, examine the performance of the method against some lower bounds. The preventive maintenance cost is

☆ This paper was financially supported by Iran National Scientific Foundation.
* Corresponding author. Fax: +98 2177240482.
*E-mail address:* sjsadjadi@iust.ac.ir (S.J. Sadjadi).

0951-8320/$ - see front matter © 2009 Elsevier Ltd. All rights reserved.
doi:10.1016/j.ress.2009.05.003

another aspect of the recent redundancy problems where maintenance is applied to a series–parallel to improve the quality of a system [8]. In this case, the proposed model is highly nonlinear and non-convex and the near-optimal solution is found using meta-heuristic approach. Samrout et al. [9] study the same preventive model using ant colony optimization and, using some numerical examples, they report promising results. In fact, there are different heuristic or meta-heuristic approaches to use to find the near-optimal solutions and, in many cases, the final solution is fairly close to optimal [10]. The reliability optimization with failure dependencies is one of other interesting works. Haiyang et al. [11] explain that, in a multi-component system, the failure of one component can reduce the system reliability in different ways such as the loss of the reliability contribution of the failed sub-system and the reconfiguration of the main system. Therefore, they introduce a new failure dependency function called redundant dependency and study the impact of the new model.

With respect to reliability and redundancy allocation, Yalaoui et al. [12] study a simple form of serial–parallel when there are lower and upper bounds for each sub-systems and the model is simplified into a traditional knapsack problem where the art of dynamic programming can be easily used to find the optimal solution.

The primary objective of this paper is to propose a heuristic method to solve a general framework of redundancy problems where there are many constraints. To the best of our knowledge, no heuristic or meta-heuristic method has been applied to this kind of problems and our contribution is the first attempt to present an efficient heuristic. Since there are $m$ linear constraints involved in the proposed model of this paper, one could build a practical case study. For instance, suppose we are interested in designing a layout for an industry, e.g. auto industry. There are different numbers of departments and each department needs, at least, one machine to work. For the sake of the simplicity, we make a simple assumption that all machines in each department are unique. For each department, various parameters such as the cost of purchasing the machines, the size of the machines, etc influence our choices. Therefore, one may expect having more than one simple constraint involved for optimal redundancy allocation. The question is to find the optimal number of unique machines for different departments such that the reliability of the factory is maximized. The other example comes from the aerospace industry when we may wish to design a space shuttle with virtually numerous limitations such as weight, cost, size, etc. As we can observe, a realistic redundancy allocation problem is generally involved with more than one single constraint and this is the main concern in this paper. The resulted problem formulation of this paper is formulated as a mixed-integer problem and the optimal solution of the problem for real-world case study is not available. Therefore, we propose an upper bound and compare the near-optimal solution with it. Besides, we propose a hybrid genetic algorithm and tune its parameters via Taguchi's method and then compare its performance with the proposed heuristic method.

This paper is organized as follows. In Section 2 we define the problem and present the necessary notations and the problem formulation. The proposed heuristic method is explained in Section 3. In Section 4 the proposed hybrid genetic algorithm is presented. In Section 5 the experimental design including data generation, Taguchi's robust design for the hybrid genetic algorithm and experimental results are presented. Finally, the conclusion remarks are given at Section 6 to summarize the contribution of the paper.

## 2. Problem statement

One of the most important types of redundancy is the serial–parallel reliability system. Fig. 1. shows the demographic of the problem we are interested. As we can observe, there are $S$ subsystems and for each subsystem we may have more than one component but with the same reliability.

The objective of this system is to determine the proper number of components in each subsystem so that the reliability is maximized subject to various constraints such as weight, expenditure, size, etc. One specific feature of this problem is that there must be at least one component in each subsystem and the number of components is not bounded from above and only the constraints can limit it. The resulted model has nonlinear objective function, some linear constraints where all variables are non-negative and integer numbers. The notations and mathematical model are presented in follow.

**Notation**

| | |
|---|---|
| $S$ | number of subsystems |
| $n_i$ | number of components used in subsystem $i$, $i = 1, \ldots, S$ |
| $R_i$ | reliability for the available components in subsystem $i$, $i = 1, \ldots, S$ |
| $A_{m \times s}$ | the coefficients of the $m$ constraints |

### 2.1. Mathematical model

The mathematical model of the problem is as follow:

$$\begin{aligned} \max \quad & z = \prod_{i=1}^{S} [1 - (1 - R_i)^{n_i}] \\ \text{subject to} \quad & An \le b \\ & n_i \ge 1, \end{aligned} \tag{1}$$

where $A \in R^{m \times S}$, $b \in R^{m \times 1}$, $n \in R^{S \times 1}$ and $n_i = 1,2, \ldots$ with $i = 1, \ldots, S$.

The objective function of (1) is the maximization of the total reliability of the serial–parallel system, and there are $m$ constraints associated with different conditions such as weight, cost, size, etc. Although the model is mixed integer and theoretically can be solved using a typical software package but a general solution procedure for this mathematical formulation is to use dynamic programming and this approach only works when there are two constraints at most. In fact, when there are more than two constraints the implementation of dynamic programming may face significant difficulties and suffers from the complexity for real-world problems. Therefore, we may look for near-optimal solutions via heuristic or meta-heuristic methods.

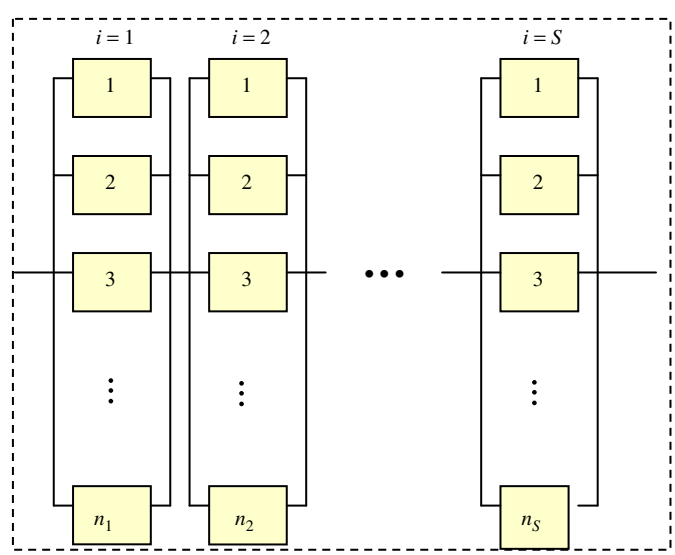


**Fig. 1.** The serial–parallel system.

## 3. The proposed heuristic

In this section, we explain the proposed heuristic. As noted earlier, the complexity of a typical serial–parallel problem is NP-Hard and we may not expect an optimal solution for large-scale problems. However, there is an optimal solution for a simpler model where there is only single constraint [13]. The primary step to find the optimal set is to calculate the following:

$$\Delta_i = \frac{\ln[1-(1-R_i)^{n_i+1}] - \ln[1-(1-R_i)^{n_i}]}{a_i}. \tag{2}$$

The procedure calculates $\Delta_i$ for each subsystem, chooses the maximum one and increase $n_i$ by one. In fact $\Delta_i$ calculates the relative change on total reliability having one extra unit of $n_i$ where $a_i$ is the coefficient of the single constraint associated with $i$th sub-system. The process continues until the available budget is assigned to the system. However, the procedure cannot be directly used for problem (1) since there is more than one constraint. The primary advantage of using (2) is the ease of calculation. As we have already mentioned, the redundancy problem is NP-Hard and it is not an easy task to develop a practical way to find the optimal solution especially where there are several constraints. However, we create an intuitive method, which uses the idea of calculating (2) to find a near-optimal solution for (1). The following pseudo-code demonstrates the implementation of the proposed method.

Procedure 1:

**Begin**
1. *Set* $R_i$,$A$,$b$, $n = [1, \ldots, 1]$;
2. ***While*** $An \le b$
3. $temp \leftarrow 0$;
4. ***for*** $i = 1$ to $S$
5. ***for*** $j = 1$ to $m$
6. $\Delta_{ji} = \frac{\ln[1-(1-R_i)^{n_i+1}]-\ln[1-(1-R_i)^{n_i}]}{a_{ji}}$.
7. ***if*** $temp < \Delta_{ji}$
8. $temp = \Delta_{ji}$;
9. $Indx = i$;
10. ***end-if***
11. ***end-for***
12. **end-for**
13. $n(Indx) = n(Indx)+1$;
14. ***end-while***

**End**.

Procedure 1 is the proposed approach to find the heuristic solution. Note that if there is only one constraint, then the final solution is also optimal. Let

$$L = \min\left[\frac{b_i}{\max a_{ij}} j = 1, \ldots, S\right] \quad i = 1, \ldots m \tag{3}$$

where [] is the largest integer number of the bracket. Then, the order of the computations for procedure 1 is $O(L.m.S)$. For small values of $L$, $m$ or $S$ the procedure runs fairly quickly on a regular computer. The optimal solution of problem (1) is not generally available for practical problems when the sizes of input parameters are greater than 15. Therefore, we need to have an upper bound in order to compare the results of our proposed method. We could use procedure 1 by considering only one constraint in order to have a good guess for optimal solution.

## 4. Hybrid genetic algorithm

The genetic algorithm is a probabilistic search method developed by Holland [14], which uses the natural or Darwinian idea of "survival of the fittest" to optimize the combinatorial problems using genetic operators such as crossover, mutation and reproduction. To optimize the proposed reliability system, a hybrid genetic algorithm (HGA) is proposed in this paper that includes two phases. In the first phase, the general GA is implemented to obtain a near-optimum solution. In the second phase, the solution obtained from the first phase is improved using the proposed heuristic method. Note that in the heuristic method initially there is only one component for each subsystem whereas here the number of components may be more than one after completion of genetic algorithm process. The main steps of the proposed hybrid GA are presented in Fig. 2.

**Algorithm** HGA

First phase:
1. **Initialize** mutation rate (*Mut.rate*), crossover rate (*Cros.rate*), mutation type (*Mut.type*), crossover type (*Cros.type*), population size (*pop.size*), generation number (*Gen.No.*), parent selection strategy;
2. Generate an initial population of solutions and evaluate it;
3. **Repeat**
4. Select the first and the second parent for mating via a specified selection strategy;
5. Perform crossover operation with the probability of crossover rate;
6. Mutate randomly selected individuals with the probability of mutation rate;
7. Transfer the high-quality and new-born individuals to the next generation;
8. **Until** (reaching to the defined maximum number of generations)
9. Return the best solution;

Second phase:
10. Apply the heuristic method to the best-resulted solution from the first phase.

**End**.

The encoding scheme for the proposed meta-heuristic algorithm is represented with a string of subsystems where each one includes at least one component. First a population of the solutions is generated randomly and then, for each generation, genetic operators are applied to the population of the current generation trying to evolve the solutions. In this part of the process, the generated solutions may violate the constraints. There are different approaches to overcome the violation such as rejecting, repairing, modifying genetic operator and penalty strategy. We apply death penalty for the infeasible solutions of the initial population and rejecting strategy for the rest of the HGA process that works well in this case.

To produce two new solutions for the next generation, two parent solutions are selected to pair. There are some frequently used strategies such as tournament, roulette wheel and queen-based selection strategies to choose a member of the population as one parent. In this paper, we test two combinations of the

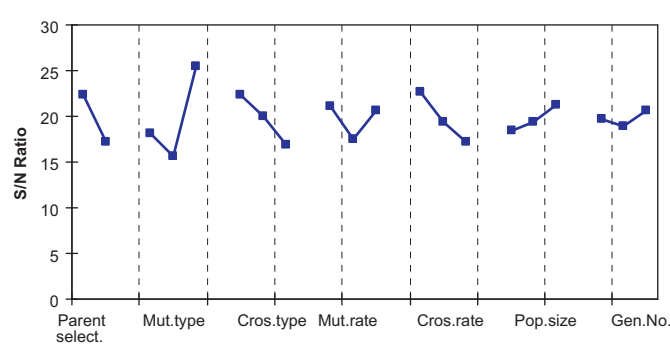


**Fig. 2.** Effect chart for $S/N$ ratio.

aforesaid strategies and choose the better one according to the following procedures.

Tournament-roulette wheel selection strategy: This strategy selects two solutions based on tournament method for selecting the first parent and roulette wheel method for the second parent. The first one, tournament strategy, chooses the winner based on the solution quality amongst pre-specified solutions whereas the last one, roulette wheel, selects the solution based on its probability in the population.

Queen-tournament selection strategy: This strategy selects two solutions where the first one is the best member of the population known as queen, whose idea comes from the queen bee, which is unique during her life and this is the same property of the best solution in the algorithm. The other one is selected according to the tournament selection discussed earlier.

Three types of crossover operator are applied in this paper to exploit the new solutions in the solution space as follows:

i. *Arithmetic crossover*
This type of crossover holds the convex concept for offsprings. If A and B are the two selected chromosomes, the produced chromosomes from crossover operation will be obtained through Eqs. (4) and (5):

$$C = \lambda A + (1-\lambda)B, \tag{4}$$

$$D = (1-\lambda)A + \lambda B, \tag{5}$$

where $\lambda$ is a value between 0 and 1. To obtain the number of components for each subsystem the value of each gene is rounded down. In this case the resulted solutions do not necessarily resemble to their parents.

ii. *Uniform crossover*:
In this kind, an offspring is generated where each of its value is inherited from each of its parents with the same probability.

iii. *Two-points crossover*
With this crossover operator, two points of crossover are randomly selected $i,j \in \{1,2,\ldots,S-1\}$, where $i<j$. The parent, defined by them, is exchanged for generating two offsprings as follows:

$$H_1 = (c_1^1, c_2^1, \ldots, c_i^2, c_{i+1}^2, \ldots, c_j^2, c_{j+1}^1, \ldots, c_S^1), \tag{6}$$

$$H_2 = (c_1^2, c_2^2, \ldots, c_i^1, c_{i+1}^1, \ldots, c_j^1, c_{j+1}^2, \ldots, c_S^2). \tag{7}$$

In this case, the value of each gene for each offspring coincides with the value of this gene in one of the parents.

Mutation operation, which is another genetic operator serves to explore new attributes that could be beneficial to the optimization process. Three types of mutation operator are applied for our proposed method as follows:

i. *Probabilistic mutation*:
In this type of mutation, each gene can be changed into (8) or (9) with equal probability.

$$a_j^* = a_j + (u_j - a_j) r \left(1 - \frac{i}{\max gen}\right), \tag{8}$$

$$a_j^* = a_j - (a_j - l_j) r \left(1 - \frac{i}{\max gen}\right), \tag{9}$$

where max*gen* is the maximum number of generations, $i$ is the number of current generation and $l_j$, $u_j$ are the maximum and the minimum values of the $j$th gene, respectively.

ii. *Non-uniform mutation (NUM)*:
Michalewicz's non-uniform mutation is one of the widely used mutation operators in real-coded GAs [15]. From a point $x^t = (x_1^t, x_2^t, \ldots, x_S{}^t)$ the mutated point $x^{t+1} = (x_1^{t+1}, x_2^{t+1}, \ldots, x_S^{t+1})$ is created as follow:

$$x_i^{t+1} = \begin{cases} x_i^t + \Delta(t, x_i^u - x_i^t) & \text{if } r \le 0.5 \\ x_i^t - \Delta(t, x_i^t - x_i^l) & \text{otherwise,} \end{cases} \tag{10}$$

where $t$ is the number of current generation and $r$ is a uniformly distributed random number between 0 and 1, $x_i{}^l$ and $x_i{}^u$ are the lower and the upper bounds for the $i$th component of the decision vector, respectively. The function $\Delta(t,y)$ given in Eq. (10) takes values in the interval $[0,y]$ and it is defined as follow:

$$\Delta(t,y) = y\left(1 - u^{(1-t/T)^b}\right), \tag{11}$$

where $u$ is a uniformly distributed random number between the interval [0,1], $T$ is the maximum number of generations and $b$ is a parameter determining the strength of the mutation operator. In the initial generations, non-uniform mutation tends to search the space uniformly and in the later generations it tends to search the space locally i.e. closer to its descendants.
Note that there is no any upper bound for the variables of the problem. Therefore, for two discussed mutation operators, we must calculate the upper limit for each gene according to the constraints and update it for each gene.

iii. *Echelon mutation*:
In this type, two genes are selected randomly to be changed. Then their values are increased or decreased by one with the same probability. Note that if there is one component in a selected gene, a randomly generated value is assigned instead of decreasing it.
Finally, the reproduction operation is applied to move some high-quality solutions into the next generation without having any change on their values.

## 5. The experimental design

A computational experiment was conducted to test the performance of the proposed heuristic and meta-heuristic implemented. The heuristic and the meta-heuristic were both coded in C++ language and ran upon the generated instances described in the following subsection.

### 5.1. Data generation

In order to examine the performance of the proposed heuristic and meta-heuristic method of this paper we have run them using various random numbers. The input parameters, $A$ and $b$ are generated using normally distributed numbers $\sigma_j$ and $\sigma_{ij}$ with $A(i,j) = 10 \times S + [10 \times \sigma_{ij}]$ and $b_j = 1200 \times S + [10 \times \sigma_j]$ where $j = 1, \ldots, m$, $i = 1, \ldots, S$. The reliability numbers are also generated randomly in the interval of $\sigma_j \in [0.5, 1.0]$ where $j = 1, \ldots, S$. The algorithms are run for different problem sizes, i.e. the number of subsystems and the number of constraints as $S = 10,15,20,25,30$ and $m = 10,15,20,25,30$, respectively. Thus, 25 scenarios were tested in this paper.

### 5.2. Taguchi method for tuning HGA parameters

As mentioned previously, various factor levels are considered in the proposed HGA where different combinations give different results. To achieve better robustness of the algorithm by not

producing functional variance under external environment influence, the parameter design developed by Taguchi in the early 1960s can be applied to process design. Taguchi method has become popular among many researches, recently. Tsai et al.[16] and Ho et al. [17] use Taguchi method on designing two-dimensional IIR digital structure-specified filters and predicting the surface roughness in end milling process, respectively.

In Taguchi method, orthogonal arrays are used to investigate a large number of parameters with a small number of experiments. Taguchi separates factors into two main groups of controllable and noise factors. The controllable factors are placed in the inner orthogonal array, and noise factors are placed in the outer orthogonal array. The measured values of the quality characteristic (the value of the response variable) obtained through the experiments are transformed into signal-to-noise ratio ($S/N$) and using some further analysis, the optimal parameter level combination can be found. Here, the term 'signal' denotes the desirable value (response variable) and 'noise' denotes the undesirable value (variance). This explains the reason why this type of parameter design is called a robust design. As well as $S/N$ ratio, which is utilized for minimizing the variances, the mean of the quality characteristics is also used to determine the adjustment factors that are utilized to approach the quality characteristic to the objective point. Generally, parameter design procedures can be explained as follows:

i. The influences of the controllable factors are evaluated over the $S/N$ ratio and the mean of response. In fact the appropriate experimental design is performed over $S/N$ ratio and the mean of the considered characteristic.
ii. For each factor that has significant impact on the $S/N$ ratio, the level that increases the $S/N$ ratio is selected.
iii. Each factor that does not have any significant impact on $S/N$ ratio, and has significant impact on the mean of responses ($y$), is considered as an adjustment factor, and the level where its mean of $y$ is closer to the objective point, is selected.
iv. The factors with no significant impact either on $S/N$ ratio or on mean of $y$, is regarded as economical factors, and levels that decrease the cost of production are selected. [18].

In the following subsections, first, the pattern for the generation of the test data and the appropriate Taguchi scheme for HGA are presented. Then Taguchi experimental design is performed. Finally, ANOVA is applied to determine the effective parameters that have significant impact on the robustness of algorithm.

Table 1 shows the parameters of HGA and their levels. To decide which combination of these factor levels results a good solution, all combination of factor levels must be tested. The full factorial design requires $2^1 \times 3^6 = 731$ experiments for selecting the appropriate parameters of HGA. With the consideration of the cost and the time, this kind of experimental design is not economical. On the other hand, theoretically, HGA does not require testing all combinations of factors. Therefore, we have used fractional replicated designs. To select the appropriate orthogonal array, the degree of freedom is calculated as: $(6 \times 2)+1 \times 1+1 = 14$. That is two degrees of freedom for each factor that has three levels, one degree of freedom for the factor that has two levels and one degree of freedom for total mean. Thus, the appropriate array at least must have fourteen rows. The proper orthogonal array is $L_{18}(2^1 \times 3^6)$, which is shown in Table 2. In Table 2 control factors are assigned to the columns of the orthogonal array and the corresponding integers in these columns indicate the actual levels of these factors. Note that for the foregoing scheme only main effects were estimated.

As mentioned previously, HGA was experimented based on the orthogonal array distribution methods. So, in HGA 18 different level-combinations of control factors were considered. Each trial consists of 25 instances running with 5 replications. To calculate the performance of each trial, the reliability of each problem is transformed into its relative percentage deviation ($RPD$) defined as follow:

$$mean(RPD)_{ij} = \frac{upper\ \ bound_i - \underset{k}{Mean}\ \ (reliability_{ijk})}{upper\ \ bound_i}, \tag{12}$$

where $i$, $j$ and $k$ denote the indices of problem, trial and replication, respectively. For each trial, the mean $RPD$ results are transformed into $S/N$ values. Since, the quality characteristic is expected performance, which prefers "the lower is better" principle, $S/N$ ratio has the characteristic of "the greater the better". Proper formula is presented below:

$$S/N\ \ \text{ratio}:\ \ \eta_j = -10\log\left(\frac{1}{N}\sum_{i=1}^{N} RPD_{ij}^2\right), \quad N = 1,\ldots,25. \tag{13}$$

For each level of control factors, the $S/N$ ratios are averaged and its value is plotted against each control factor in Fig. 2.

To test the statistical significance of the factors, ANOVA is carried out. First, the sum square of each factor (SSX) is calculated as shown in Table 3. Since the sum of SSX for *Pop.size* and *Gen.No.*, is trivial compared with SSE, they can be pooled in error term. Therefore, the pooled ANOVA table presented in Table 4 is achieved. We study ANOVA under five percent significance level. So, we compare $F$ statistic of each factor with $F(1,8,5\%) = 5.32$ or $F(2,8,5\%) = 4.46$, which are extracted from $F$ statistical table.

**Table 1**
Factor levels in HGA.

| Factors | Levels | | |
|---|---|---|---|
| | 1 | 2 | 3 |
| *Gen.No.* | 500 | 800 | 1000 |
| *Pop.size* | 300 | 600 | 900 |
| *Mut.type* | Probabilistic | NUM | Echelon |
| *Cros.type* | Uniform | Arithmetic | Two point |
| *Mut.rate* | 0.03 | 0.05 | 0.09 |
| *Cros.rate* | 0.8 | 0.85 | 0.9 |
| Parent selection | Tournament-roulette wheel | Queen-tournament | |

**Table 2**
The orthogonal array $L_{18}(2^1 \times 3^6)$.

| Trial | Gen.No. | Pop.size | Cros.rate | Mut.rate | Cros.type | Mut.type | Parent selection |
|---|---|---|---|---|---|---|---|
| 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| 2 | 2 | 2 | 2 | 2 | 2 | 1 | 1 |
| 3 | 3 | 3 | 3 | 3 | 3 | 1 | 1 |
| 4 | 3 | 2 | 2 | 1 | 1 | 2 | 1 |
| 5 | 1 | 3 | 3 | 2 | 2 | 2 | 1 |
| 6 | 2 | 1 | 1 | 3 | 3 | 2 | 1 |
| 7 | 2 | 3 | 1 | 2 | 1 | 3 | 1 |
| 8 | 3 | 1 | 2 | 3 | 2 | 3 | 1 |
| 9 | 1 | 2 | 3 | 1 | 3 | 3 | 1 |
| 10 | 2 | 2 | 3 | 3 | 1 | 1 | 2 |
| 11 | 3 | 3 | 1 | 1 | 2 | 1 | 2 |
| 12 | 1 | 1 | 2 | 2 | 3 | 1 | 2 |
| 13 | 3 | 1 | 3 | 2 | 1 | 2 | 2 |
| 14 | 1 | 2 | 1 | 3 | 2 | 2 | 2 |
| 15 | 2 | 3 | 2 | 1 | 3 | 2 | 2 |
| 16 | 1 | 3 | 2 | 3 | 1 | 3 | 2 |
| 17 | 2 | 1 | 3 | 1 | 2 | 3 | 2 |
| 18 | 3 | 2 | 1 | 2 | 3 | 3 | 2 |

The results show that the mutation type and the parent selection strategy with 37.47614 percent and 13.08457 percent have significant impact on the algorithm's performance, respectively. Therefore we consider them as two control factors and from Fig. 2 the level, which has the maximum $S/N$ ratio is selected for them, i.e. *Tournament-roulette wheel* for parent selection strategy and *Echelon mutation* for mutation type.

To know about the adjustment factors, the mean of *RPD* for reliability is used. For each of the 25 instances the mean of *RPD* value in each trial is calculated as follow:

$$mean(RPD)_{ij} = \frac{\max_k(reliability_{ijk}) - \underset{k}{mean}(reliability_{ijk})}{\max_k(reliability_{ijk})}, \tag{14}$$

where $i$, $j$ and $k$ denote the indices for problem, trial and replication, respectively. Then the mean of *RPD* values of the 25 instances are averaged to obtain the *RPD* results of reliability in each trial. Then for each control factor the mean *RPD* are averaged for each level and its value is plotted in Fig. 3.

For statistical significance test of factors on the *RPD* values, ANOVA is carried out. First, the sum square of factors for *RPD* values are calculated as shown in Table 5. Since the sum of SSX for *Pop.size* and *Gen.No.*, is trivial compared with SSE, they can be pooled in error term. Therefore, the pooled ANOVA table presented in Table 6 is achieved. $F$ statistic for each factor is compared with $F(1,8,5\%) = 5.32$ or $F(2,8,5\%) = 4.46$ and the results show that the mutation type, the parent selection strategy, the mutation rate and the crossover rate with 23.67428 percent, 12.27777 percent, 15.26506 percent and 18.14142 percent have significant impact on the algorithm's performance, respectively. Since the mutation type and the parent selection strategy are considered as control factors, just the mutation rate and the crossover rate are considered as adjustment factors and from Fig. 3 their proper levels are chosen as 0.09 and 0.8, respectively.

Other factors that have insignificant impact either on $S/N$ ratio or on *RPD* values are the economical factors and the level that results a high-quality solution or decrease the CPU time could be selected. Since both of these factors are desirable, we chose the middle level of *Pop.size* and *Gen.No.*, i.e. 600 and 800, respectively. Also *Uniform crossover* is opted as crossover type.

**Table 3**
Sum square of factors.

| Factor | *Gen.No.* | *Pop.size* | *Mut.type* | *Cros.type* | *Mut.rate* | *Cros.rate* | Parent selection |
|---|---|---|---|---|---|---|---|
| SSX | 7.535633 | 24.42231 | 314.816 | 88.66829 | 46.31059 | 93.48742 | 116.0111 |
| SSE | | | | 40.9838 | | | |
| SST | | | | 732.2351 | | | |

**Table 4**
ANOVA for $S/N$ ratio.

| Factors | *Df* | *SS* | *MS* | *F* | Percent *X* |
|---|---|---|---|---|---|
| Parent selection | 1 | 116.0111 | 116.0111 | 5.742768 | 13.08457 |
| *Mut.type* | 2 | 314.816 | 157.408 | 7.791991 | 37.47614 |
| *Cros.type* | 2 | 88.66829 | 44.33414 | 2.194623 | 6.591568 |
| *Mut.rate* | 2 | 46.31059 | 23.1553 | 1.146231 | 0.806856 |
| *Cros.rate* | 2 | 93.48742 | 46.74371 | 2.313901 | 7.249708 |
| Error | 8 | 161.61 | 20.20125 | | 34.79115 |
| Total | 17 | 732.2351 | 43.07265 | | |

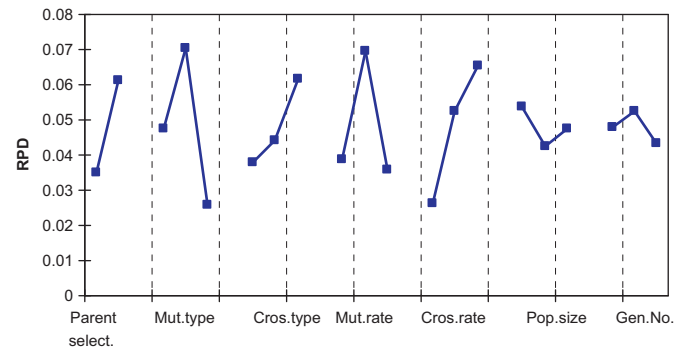


**Fig. 3.** Effect chart for *RPD* value.

**Table 5**
Sum square of factors for *RPD* values.

| Factor | *Gen.No.* | *Pop.size* | *Mut.type* | *Cros.type* | *Mut.rate* | *Cros.rate* | Parent selection |
|---|---|---|---|---|---|---|---|
| SSX | 0.000246 | 0.000393 | 0.005969 | 0.001793 | 0.004161 | 0.00478 | 0.003073 |
| SSE | | | | 0.001084 | | | |
| SST | | | | 0.0215 | | | |

**Table 6**
ANOVA for RPD values.

| Factors | *Df* | *SS* | *MS* | *F* | Percent *X* |
|---|---|---|---|---|---|
| Parent selection | 1 | 0.003073 | 0.003073 | 6.989816 | 12.24777 |
| *Mut.type* | 2 | 0.005969 | 0.002985 | 6.788996 | 23.67428 |
| *Cros.type* | 2 | 0.001793 | 0.000897 | 2.03975 | 4.252088 |
| *Mut.rate* | 2 | 0.004161 | 0.002081 | 4.732717 | 15.26506 |
| *Cros.rate* | 2 | 0.00478 | 0.00239 | 5.436064 | 18.14142 |
| Error | 8 | 0.003517 | 0.00044 | | 26.41939 |
| Total | 17 | 0.0215 | 0.001265 | | |

### 5.3. Experimental results

An experiment was conducted to test the performance of the proposed heuristic with HGA. Each algorithm was run on a PC with a Pentium IV 3 GHz processor with 2 GB of RAM. The results of running the proposed heuristic and HGA are presented in Table 7. Note that, HGA was run regarding to the optimum level of its parameters. As can be seen there is no significant difference between the results obtained from the proposed heuristic and HGA. To statistically analyze the performance of the proposed heuristic, the average and the maximum reliability resulted from HGA meta-heuristic, Kruskal–Wallis procedure as a non-parametric method is carried out to test the following hypothesis:

$$\begin{cases} H_0 : \mu_{heuristic} = \mu_{mean_HGA} = \mu_{\max_HGA} \\ H_1 : \text{not all } \mu\text{'s are equal} \end{cases},$$

where $\mu_i$ is the mean and $i$ = heuristic, mean_HGA, and max_HGA.

*Test statistic*:

$$H = \frac{12}{n(n+1)}\sum_{i=1}^{k}\frac{R_i^2}{n_i} - 3(n+1)$$
$$= \frac{12}{75(76)}\left[\frac{950^2}{25} + \frac{934^2}{25} + \frac{966^2}{25}\right] - 3(76) = 0.0431158$$

**Table 7**
Reliability and CPU time of the proposed heuristic and HGA.

| Problem set | Problem size | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| | No. of constraints | No. of subsystems | Upper bound | Heuristic | CPU time | HGA | | CPU time |
| | | | | | | Mean | Max | |
| 1 | 10 | 10 | 0.99995 | 0.999857 | 0.015 | 0.999856 | 0.999857 | 5.188 |
| 2 | 10 | 15 | 0.99281 | 0.992376 | 0.015 | 0.992339 | 0.992388 | 6.750 |
| 3 | 10 | 20 | 0.90261 | 0.895548 | 0.016 | 0.895548 | 0.895548 | 8.031 |
| 4 | 10 | 25 | 0.81186 | 0.805469 | 0.015 | 0.803413 | 0.805469 | 9.190 |
| 5 | 10 | 30 | 0.46387 | 0.450441 | 0.015 | 0.448567 | 0.450441 | 10.406 |
| 6 | 15 | 10 | 0.99944 | 0.999438 | 0.015 | 0.999469 | 0.999469 | 5.109 |
| 7 | 15 | 15 | 0.96172 | 0.954524 | 0.016 | 0.954466 | 0.954887 | 6.047 |
| 8 | 15 | 20 | 0.85859 | 0.851829 | 0.031 | 0.852151 | 0.852231 | 8.031 |
| 9 | 15 | 25 | 0.59106 | 0.591057 | 0.031 | 0.590104 | 0.591057 | 9.563 |
| 10 | 15 | 30 | 0.38962 | 0.377052 | 0.031 | 0.378044 | 0.382890 | 10.796 |
| 11 | 20 | 10 | 0.9999 | 0.999812 | 0.016 | 0.999806 | 0.999812 | 5.422 |
| 12 | 20 | 15 | 0.9737 | 0.962404 | 0.016 | 0.962233 | 0.962404 | 6.922 |
| 13 | 20 | 20 | 0.9694 | 0.963717 | 0.031 | 0.965418 | 0.965607 | 8.516 |
| 14 | 20 | 25 | 0.81282 | 0.793767 | 0.031 | 0.791207 | 0.793767 | 9.594 |
| 15 | 20 | 30 | 0.34207 | 0.320353 | 0.031 | 0.324375 | 0.331031 | 10.50 |
| 16 | 25 | 10 | 0.99993 | 0.999842 | 0.031 | 0.999840 | 0.999848 | 5.359 |
| 17 | 25 | 15 | 0.9953 | 0.994257 | 0.031 | 0.993984 | 0.994257 | 6.859 |
| 18 | 25 | 20 | 0.90078 | 0.883253 | 0.047 | 0.883253 | 0.883253 | 8.094 |
| 19 | 25 | 25 | 0.68881 | 0.688812 | 0.047 | 0.685695 | 0.688812 | 10.563 |
| 20 | 25 | 30 | 0.54114 | 0.52264 | 0.062 | 0.522640 | 0.522640 | 11.203 |
| 21 | 30 | 10 | 0.99998 | 0.999976 | 0.031 | 0.999968 | 0.999976 | 5.578 |
| 22 | 30 | 15 | 0.98009 | 0.966725 | 0.047 | 0.965163 | 0.966725 | 7.438 |
| 23 | 30 | 20 | 0.90249 | 0.872652 | 0.047 | 0.872652 | 0.872652 | 8.859 |
| 24 | 30 | 25 | 0.78223 | 0.761868 | 0.063 | 0.761868 | 0.761868 | 10.218 |
| 25 | 30 | 30 | 0.48779 | 0.463535 | 0.063 | 0.463518 | 0.463535 | 11.437 |

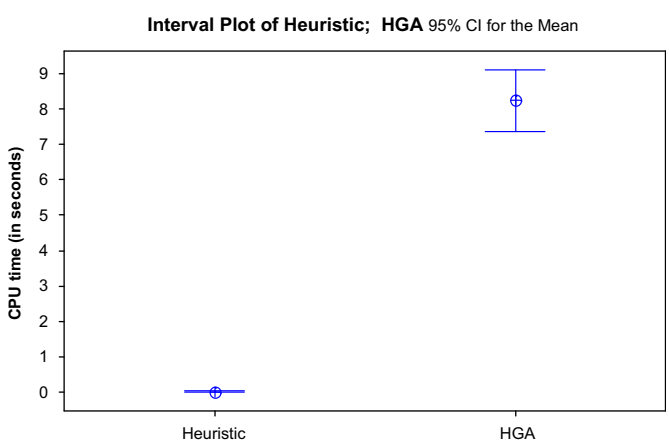


**Fig. 4.** Interval plot for the CPU time.

Since $H < \chi^2_{0.05,2} = 5.991$, we cannot reject the null hypothesis and we can conclude that there is no difference amongst algorithms.

One striking feature of the proposed heuristic algorithm is that it could result a high-quality solution in a reasonable amount of time. Fig. 4 demonstrates the difference between the CPU times for two methods.

Therefore, considering the CPU time this heuristic is more efficient than HGA and it is applicable for redundancy problems with large number of constraints. To have a sense from the optimal solution and test how much the proposed heuristic and HGA are close to the optimal solution, we consider an upper bound when only the first constraint is considered. As it is shown in Table 7 there is a minor difference among the results.

## 6. Conclusions

We have presented a general framework for redundancy problem. The proposed method of this paper is capable of solving many large-scale problems where there are different subsystems and constraints. Such a model is believed to be used for design problems in many industries such as aerospace, layout facilities, etc. The mathematical modeling of the proposed method is NP-Hard and the optimal solution is not a practical approach. Therefore, we have proposed heuristic and meta-heuristic approaches to find some practical solutions. We have shown through some numerical examples that the proposed heuristic can result a high-quality solution in a reasonable amount of time. The other advantage of the proposed method is that it does not need any special parameter setting and it always acts in a feasible region and remains feasibility. The performance of the proposed method was compared with a meta-heuristic approach that was developed in this paper.

## Acknowledgment

The authors would like to thank the anonymous referees for their constructive comments on the earlier version of the paper.

## References

[1] Elegbede AOC, Chu C, Adjallah KH, Yalaoui F. Reliability allocation through cost minimization. IEEE Transaction on Reliability 2003;52:106–11.
[2] Yalaoui A, Chu C, Chatelet E. Reliability allocation problem in series–parallel system. Reliability Engineering and System Safety 2005;90:55–61.
[3] Gen M, Yun YS. Soft computing approach for reliability optimization: state-of-the-art survey. Reliability Engineering System Safety 2006;91:1008–26.
[4] Misra KB. Reliability optimization of series–parallel system. IEEE Transaction on Reliability 1972;21:230–8.
[5] Ramirez-Marquez JE, Coit DW, Konak A. Redundancy allocation for series–parallel systems using a max–min approach. IIE Transaction 2004;36:891–8.

[6] Ramirez-Marquez JE, Coit DW. A heuristic for solving the redundancy allocation problem for multi-state series–parallel systems. Reliability Engineering and System Safety 2004;83:341–4.
[7] Tavakkoli-Moghaddam R, Safari J, Sassani F. Reliability optimization of series–parallel systems with a choice of redundancy strategies using a genetic algorithm. Reliability Engineering and System Safety 2008;93: 550–6.
[8] Bris R, Châtlet E, Yalaoui F. New method to minimize the preventive maintenance cost of series–parallel systems. Reliability Engineering and System Safety 2003;82:247–55.
[9] Samrout M, Yalaoui F, Châtlet E, Chebbo N. New methods to minimize the preventive maintenance cost of series–parallel systems using ant colony optimization. Reliability Engineering and System Safety 2005;89:346–54.
[10] You P, Chen T. An efficient heuristic for series–parallel redundant reliability problems. Computers and Operations Research 2005;32:2117–27.
[11] Yu H, Chu C, Châtlet E, Yalaoui F. Reliability optimization of a redundant system with failure dependencies. Reliability Engineering and System Safety 2005;92:1627–34.
[12] Yalaoui A, Châtlet E, Chu C. A new dynamic programming method for reliability & redundancy allocation in a parallel–series system. IEEE Transaction on Reliability 2005;54:254–61.
[13] Wallace R, Blischke L, Prabhakar Murthy DN. Reliability: modeling, prediction, and optimization. 1st ed. Wiley-Interscience; 2000. p. 832.
[14] Holland JH. Adaptation in natural and artificial systems. The University of Michigan Press; 1975.
[15] Michalewicz Z. Genetic algorithms+data structures = evolution programs. New York: Springer; 1992.
[16] Jinn-Tsong Tsai, Wen-Hsien Ho, Jyh-Horng Chou. Design of two-dimensional IIR digital structure-specified filters by using an improved genetic algorithm. Expert Systems with Applications 2009;36(3, Part 2):6928–34.
[17] Wen-Hsien Ho, Jinn-Tsong Tsai, Bor-Tsuen Lin, Jyh-Horng Chou. Adaptive network-based fuzzy inference system for prediction of surface roughness in end milling process using hybrid Taguchi–genetic learning algorithm. Expert Systems with Applications 2009;36(2, Part 2):3216–22.
[18] Phadke MS. Quality engineering using robust design. Upper Saddle River, NJ: Prentice-Hall; 1989.